

Music Bot Documentation

Discord multi-worker music bot: `main.py` controller, `worker.py` workers, `db.py` storage and `appsettings.py` settings.

This document explains every function, class and method of the source files, the settings you can change, the rules for who can control playback, the saved-songs ("algo") and autoplay system, and the messages the controller and the workers exchange.

Contents

1. [Architecture overview](#)
2. [Configuration](#)
3. [Commands](#)
4. [Who can control playback](#)
5. [Saved songs \(algo\) and autoplay](#)
6. [main.py reference \(controller\)](#)
7. [worker.py reference \(worker\)](#)
8. [db.py and appsettings.py reference](#)
9. [IPC operations and state events](#)
10. [Tunable constants and files](#)
11. [Graceful restart](#)
12. [Deployment and service](#)

1. Architecture overview

The system has four parts that run together on the server:

Part	File / service	Job
Controller	<code>main.py</code> (systemd unit <code>SERVER_musicbots</code>)	Logs in as the main bot, registers the slash commands for every server, enforces the whitelist, reads the dev <code>prefix</code> commands, starts and watches the workers, decides which worker serves which request, and relays commands to them.
Workers	<code>worker.py</code> , started once per worker token with <code>--index N</code>	Each copy is a separate Discord bot in its own process. Worker N uses <code>tokens[N]</code> . A worker keeps one session per server , so it can play in one voice channel in every server at the same time. It searches, resolves streams, plays, records listens and runs autoplay.
Storage	<code>db.py</code> → <code>database/</code>	SQLite files. <code>database/server/servers.db</code> holds per-server settings; <code>database/algo/<server_id>.db</code> holds every user's saved songs for that server.
Lavalink	<code>localhost:2333</code> (unit <code>SERVER_lavalink</code>)	The audio server, with the youtube-plugin and lavasrc-plugin. Workers connect to it with wavelink. YouTube streams are resolved by yt-dlp first and given to Lavalink as plain URLs.

How a play request flows

1. A user runs `/play` in Discord. The controller checks the server is whitelisted and the control channel (`_guard`).
2. The controller takes the pick lock, asks every worker for its real state (`sync`), and chooses a worker and voice channel (`_pick_for_play`). A new worker is *reserved* for that user before the lock is released, so two simultaneous `/play`s never get the same worker.
3. The controller sends `search_and_play` to that worker over its Unix socket `.worker<N>.sock`, including the user id.
4. The worker creates a session for that server (the user becomes its **controller**), searches (Spotify for clean metadata, YouTube Music for candidates), scores the candidates, connects to the voice channel and plays the best one. YouTube tracks are resolved to a direct stream URL with yt-dlp. The worker sets the voice channel status to `🎵Playing - <title>`.
5. The worker replies with the track details and a session snapshot; the controller turns them into an embed. Later track changes are announced by the worker itself in the text channel.
6. Whenever a session starts, changes or ends, the worker sends a `state` event to the controller over `.main.sock`, so the controller's view of who is busy stays correct even when a worker leaves on its own (idle timeout, kicked, failures).

Discord users never talk to the workers directly. All user input goes through the controller.

2. Configuration

There are three places settings live:

Where	Scope	Edited by
<code>server_settings.json</code>	Whole bot (every server)	You, on the server. Contains secrets, git-ignored.
<code>.env</code>	Whole bot	You, on the server. <code>GRQQ_API_KEY</code> for the title filters and AI autoplay fallback.
<code>database/server/servers.db</code>	One server	Server admins, with <code>/settings</code> in Discord.

server_settings.json

Copy `server_settings.json.example` and fill it in. Behaviour keys are re-read automatically when the file changes (no restart needed). `tokens` and `lavalink` are only read at start-up, so changing them needs a restart.

Key	Default	Meaning
<code>tokens</code>	— (required)	<code>tokens[0]</code> is the main controller bot, <code>tokens[1]</code> , <code>tokens[2]</code> , ... are the workers. One worker process is started per extra token. At least 2 tokens are required.
<code>lavalink.uri</code>	— (required)	Lavalink address, e.g. <code>http://localhost:2333</code> .
<code>lavalink.password</code>	—	Lavalink password.
<code>dev_ids</code>	<code>[]</code>	Discord user ids of the bot developers. They can use the <code>prefix</code> commands in any server and always control playback.
<code>whitelist_enabled</code>	<code>true</code>	When true, the bot only works in servers listed in <code>whitelist</code> .
<code>whitelist</code>	<code>[]</code>	Server ids the bot may be in.
<code>leave_message</code>	"This server is not currently whitelisted, bot will not function"	Posted (with the server owner pinged) before the bot leaves a non-whitelisted server.
<code>algo_max_kb</code>	<code>1024</code>	Storage cap for one user's saved songs on one server, in KB. There is no song-count limit; a saved song takes about 100 bytes, so 1 MB is roughly 10,000 songs.
<code>autoplay_seed_count</code>	<code>5</code>	How many songs autoplay looks at when choosing what to play next.

Per-server settings (/settings)

Stored in `database/server/servers.db`, one row per server. Change them with the `/settings` panel (section 3). `get-env` shows them.

Key	Default	Meaning
<code>CC_ID</code>	not set	Control channel. Slash commands only work there and workers post their now-playing messages there. <code>/settings</code> itself works in any channel, so a wrong value cannot lock admins out.
<code>VCW<n></code>	not set	Fixed voice channel of worker n. A worker with this setting always joins that channel and is not borrowed for other channels. A worker without it joins the voice channel of the person who used the command.

Key	Default	Meaning
SRC	sp	Default search source for <code>/play</code> and <code>/playlist</code> : <code>sp</code> Spotify, <code>yt</code> YouTube Music, <code>sc</code> SoundCloud. The command's <code>source</code> option overrides it.
P_EPH	false	<code>/play</code> replies private (only the person who ran it sees them).
PL_EPH	false	<code>/playlist</code> replies private.
AP_EPH	false	<code>/autoplay</code> replies private.
HC_EPH	false	<code>/hctest</code> replies private.
CC_EPH	false	<code>/control</code> panel and its button feedback private.
S_EPH	false	<code>/stop</code> replies private.

When playback is started with a private command (`P_EPH` , `PL_EPH` or `AP_EPH` on), the worker also stops posting "Now Playing" messages on track changes, and the "Playback failed" notice. Those are normal channel messages, which Discord can't make private. The current song still shows in the voice channel status and in `/control` . The setting follows the most recent play command on that worker.

Discord permissions the bots need

- **Main bot:** Message Content Intent enabled in the Developer Portal (for the `prefix` commands). Send Messages in the server's first channel (for the whitelist notice).
- **Workers:** Connect and Speak in voice channels, Send Messages in the control channel, and **Set Voice Channel Status** (to show the song title on the channel; without it playback still works and a log line notes the missing permission).
- Every server needs the main bot **and** all worker bots invited.

3. Commands

Slash commands

Command	Who	What it does
<code>/play query [source] [worker]</code>	Anyone (subject to section 4)	Plays a song by name or URL, or queues it if your worker is already playing.
<code>/playlist query [source] [worker]</code>	Anyone (subject to section 4)	Plays or queues a whole playlist (Spotify, YouTube, SoundCloud, Apple Music).
<code>/autoplay [worker]</code>	Anyone with saved songs	Starts music picked from your saved songs on this server, without a query. If you already control a playing worker, it switches that worker's autoplay to your saved songs.
<code>/stop [worker]</code>	Controller, devs, Manage Server	Stops the worker and makes it leave.
<code>/control [worker]</code>	Controller (or anyone in "All" mode), devs	Opens the playback control panel.
<code>/hctest</code>	Anyone	Health check of the controller and every worker.
<code>/settings</code>	Manage Server or dev	Opens the per-server settings panel (only you can use it).
<code>/purge [limit]</code>	Manage Messages or dev	Deletes bot messages among the last <code>limit</code> (1–100, default 20) messages in the channel.

The `worker` option forces a specific worker (0 = automatic). It is needed to use a worker fixed to a channel you are not in.

Control panel (/control)

Row	Contents
0	10s, 5s, 5s, 10s, Loop
1	Backward (restart track), Pause/Play, Stop, Skip, Mute (hides now-playing messages)
2	Queue dropdown (25 per page; marks autoplay picks)
3	Jump To, Remove, Clear selection, Autoplay, Control: Me / Control: All
4	Page buttons (only for queues longer than one page)

The panel stops working after 15 minutes; run `/control` again.

Settings panel (/settings)

Row	Contents
0	Control channel picker (empty = any channel)
1	Worker picker (choose which worker's fixed channel to edit)
2	Fixed voice channel picker for that worker (empty = follow the user)
3	Private replies multi-select (P/PL/AP/HC/CC/S)

Row	Contents
4	 Source (cycles Spotify → YouTube Music → SoundCloud), Clear control channel, Clear worker channel, Done

Changes save immediately. A voice channel already fixed to another worker is refused.

Dev prefix commands

Typed as a message wrapped in two backticks on each side. Only users in `dev_ids` can use them, in any server; other people get no reply.

Command	Effect
<code>restart</code>	Restarts all workers (music stops in every server).
<code>restart 2</code>	Restarts worker 2 only.
<code>restart full</code>	<code>systemctl restart SERVER_musicbots</code> (controller and all workers).
<code>shutdown</code>	Shuts down all workers; they stay down until <code>restart</code> .
<code>shutdown 2</code>	Shuts down worker 2 only.
<code>shutdown full</code>	<code>systemctl stop SERVER_musicbots</code> . Bring it back on the server with <code>systemctl start</code> .
<code>get-env</code> (or <code>show-env</code>)	Shows this server's settings, whitelist state and each worker's state here.

4. Who can control playback

- Whoever starts playback on a free worker (with `/play`, `/playlist` or `/autoplay`) becomes that session's **controller**. The session starts in **Me** mode.
- In **Me** mode, only the controller can play, queue, use the panel or stop. Anyone else in that voice channel is told the worker is controlled by the controller and to join another voice channel to get their own worker. They cannot even add songs to the queue.
- The controller (or a dev) can switch the panel's  /  button to **All**, which lets everyone control that worker. Only the controller (or a dev) can switch it back.
- **Devs** (`dev_ids`) can always control any session.
- Members with **Manage Server** can always **Stop** a session (with `/stop` or the panel's Stop button), even in Me mode, so a session can't get stuck if its controller leaves.
- When the session ends (stop, idle timeout, kicked, failures), the worker is free again and the next user to start playback on it becomes the new controller.

5. Saved songs (algo) and autoplay

Saving

- A song is saved for the user who **requested** it (autoplay picks are credited to the session's controller, or to the `/autoplay` user).
- A song counts as a **listen** once it finished or was played for at least 30 seconds (`PLAY_COUNT_MS`). The first listen adds it; later listens increase its play count.
- A **skip in under 30 seconds** adds a skip to a song that is already saved. Unsaved songs are not added by skips.
- $\text{Score} = \text{plays} - 0.5 \times \text{skips}$.
- There is no limit on the number of saved songs; only storage is capped (`algo_max_kb`). When a user goes over it, the songs with the **lowest score** are removed first, oldest last-play breaking ties ("most unused", not "oldest"). The song that was just played is never the one removed.
- Each server has its own file `database/algo/<server_id>.db`, holding every user's saved songs for that server.
- Songs are keyed by their YouTube video id when known (stable across Spotify/YouTube lookups), else their URL.

Saved songs influence search

When you `/play` something, your saved songs on that server nudge which result is picked:

- A result that is an upload you have already listened to gets +25 (+2 per play, up to +10 more).
- A result by an artist you play a lot gets up to +20 (grows with how often you play them).
- Both stay below one matching query word (+40), so they break ties between equally good matches but never override what you searched for.
- The AI pick is told which results you've played or whose artist you often play.
- If you have a saved upload of the exact song that was picked, that upload is played (the one you know).

Autoplay from `/control` (🎮 button)

- When the queue runs out, the worker picks songs related to the **last few songs played in this session** (`autoplay_seed_count`), whether they came from `/play` or a playlist.
- It queues 2 picks at a time, in the background while the last song is still playing, so there is no gap.
- Songs you queue yourself always go **ahead** of autoplay picks.
- Turning autoplay off removes the queued autoplay picks.
- It needs at least one song played in the session first.

`/autoplay`

- Seeds from **your saved songs** instead of what was played. Better-scored songs are more likely to be used as seeds.
- About 30% of the time (`ALGO_DIRECT_CHANCE`) it replays one of your saved songs directly; otherwise it plays something related to them.
- Errors if you have no saved songs yet on this server.

How picks are found

1. **YouTube Mix:** for a seed song, the worker loads YouTube's own radio mix (`watch?v=ID&list=RDID`), which returns about 25 related songs. One random song from the top 8 usable ones is taken per seed.

2. Picks are filtered: nothing already played in the session or already queued (by video id and by cleaned-up title, so a different upload of the same song is skipped), no altered versions (sped up, slowed, remix, cover, ...), nothing shorter than 1 minute or longer than 10 minutes.
3. **AI fallback:** if no Mix gives anything, the Groq model is asked for 5 similar songs ("Title - Artist"), and each is searched with the normal search pipeline.

6. main.py reference (controller)

Module level: `TOKENS` comes from `server_settings.json` through `appsettings.require_startup()` (the program exits with a clear message if tokens or Lavalink are missing). `EPH_KEYS` lists the private-reply keys and the command each controls. `SOURCES` maps `sp / yt / sc` to their names. `_workers` maps worker index to its `WorkerProcess`. `_pick_lock` serialises worker picking. `WORKER_SCRIPT` is `worker.py`; `MAIN_SOCKET` is `.main.sock`.

Per-server config helpers

def _gcfg(guild_id) Returns the server's settings dict (`cc_id`, `vcw`, `eph`, `src`), loading it from the server DB the first time and caching it in memory.

def _save_gcfg(guild_id) Writes the cached settings of a server back to the server DB.

def _cfg_cc(guild_id) Returns the server's control channel id, or None.

def _cfg_vcw(guild_id, index) Returns worker `index`'s fixed voice channel in that server, or None.

def _cfg_src(guild_id) Returns the server's default search source (`sp`, `yt` or `sc`), `sp` when unset or invalid.

def _eph(guild_id, key) Returns whether replies for a key such as `P_EPH` are private in that server. Unset keys return False (public).

def _migrate_legacy_settings() One-time import of the old single-server `settings.json`. Its `CC_ID`, `VCW` and private-reply flags are stored under its `G_ID` in the server DB (unless that server already has settings), then the file is renamed to `settings.json.migrated`.

Worker process handle

class WorkerProcess One worker subprocess. Besides the process handle and socket path it keeps the controller's view of that worker: `sessions` (server id → session snapshot: `channel_id`, `controller_id`, `mode`, `autoplay`, `text_channel_id`), pending reservations, the servers the worker bot is in (`guild_ids`), its bot user id and Lavalink state.

- **__init__(self, index)** — Stores the index, the socket path `.worker<index>.sock` and empty state.
- **session(self, guild_id)** — The worker's session in that server: the confirmed snapshot, else a reservation that hasn't expired (`PENDING_TTL`, 90 s), else None.
- **busy_in(self, guild_id)** — True when the worker has a session (or reservation) in that server.
- **reserve(self, guild_id, channel_id, user_id)** — Marks the worker as taken in that server by that user while a `/play` is in flight, before the worker confirms.
- **apply(self, guild_id, snap)** — Stores a session snapshot from the worker, or removes it when `snap` is None.
- **clear_state(self)** — Forgets all sessions and reservations (used when the worker restarts or dies).
- **start(self)** — Deletes a stale socket file, then spawns `python -u worker.py --index N` with its output piped to the controller, in its own process group. Starts `_stream_output` and `_reap`.
- **_stream_output(self)** — Prints each line of the worker's output prefixed with `[Worker N]`, so it shows in `journalctl`.
- **_reap(self)** — Waits for the process to exit in a thread so it doesn't become a zombie.
- **is_alive(self)** — True when the process exists and hasn't exited.
- **terminate(self, timeout=8.0)** — Stops the worker in three steps: ask it to run `shutdown_graceful` (up to 6 s), `SIGTERM` the process group and wait, then `SIGKILL`.
- **restart(self)** — Clears its state and shutdown flag, terminates it, waits 1 s and starts it again.

- `send(self, cmd, timeout=15.0)` — Sends one JSON command to the worker's socket and returns the JSON reply. Connecting is retried up to 3 times. Never raises: timeouts, empty replies and other failures become `{"status": "error", ...}`.

Worker state

`def _ordered_workers()` All workers sorted by index.

`def _worker_for_channel(guild_id, channel_id)` The worker that has a session in that voice channel of that server, or None.

`async def _resync_worker(w, timeout=4.0)` Asks a worker for its real state (`sync`) and replaces the controller's copy: servers it is in, bot user id, Lavalink state and all sessions. Returns False if the worker doesn't answer. The worker is the source of truth for busy/free.

`async def _resync_all()` Resyncs every worker at once (3 s timeout each).

`async def _handle_event(reader, writer)` Handles one `state` event from a worker (one JSON line) and applies the session snapshot it carries.

`async def _event_server()` Starts the Unix socket server on `.main.sock` that receives worker state events.

Embeds and permission helpers

`def _embed_from_response(resp)` Turns a worker reply into an embed using its `embed_type`: `playing / queued` (linked title, artist, thumbnail, duration, source, found-via, queue position) or `playing_playlist / queued_playlist` (name, track count, source, skipped local files, artwork). None when there is no `embed_type`.

`def _simple_embed(title, colour)` An embed with only a title and colour.

`def _err_embed(msg)` A red embed whose title is the message prefixed with **✖**.

`def _all_busy_embed(guild_id)` The red "No bots available" embed, counting busy workers among those in this server.

`def _is_dev(user)` True when the user is in `dev_ids`.

`def _has_perm(user, perm)` True when the member has a server permission such as `manage_guild`.

`def _allowed(session, user)` The control rule: True for devs, the session's controller, or anyone when the session is in All mode.

`def _locked_msg(w, session)` The "🔒 Worker N is controlled by @X. Join another voice channel to get your own worker." text.

`def _message_embed(resp)` Turns a short worker reply (`stopped` , `paused` , `skipped` , `autoplay_on` , `mode_all` , `seeked` , ...) into a feedback embed using `_MESSAGE_EMBEDS`. None for unknown messages.

Main bot, whitelist and events

`class MainBot(discord.Client)` The controller's Discord client with the slash-command tree.

- `__init__(self)` — Default intents plus `message_content` (for the `prefix` commands).
- `setup_hook(self)` — Runs once at start-up. Syncs the slash commands **globally** (every server), then logs in briefly with every worker token to clear any slash commands registered on the worker applications.

`async def _enforce_whitelist(guild)` Returns True for a whitelisted server. Otherwise it posts `leave_message` with the server owner pinged in the system channel (or the first text channel it can write in, so it never spams every channel), leaves the server and returns False.

`async def on_ready()` Logs the login, enforces the whitelist for every server the bot is in, and on the first ready removes leftover per-server slash commands from the old single-server version (so commands don't show twice).

`async def on_guild_join(guild)` Enforces the whitelist when the bot is added to a new server.

Dev prefix commands

`def _env_summary(guild)` The `get-env` text: whitelist state, `CC_ID`, `SRC`, each worker's `VCW<n>` with its state here (busy, free or not in this server), and every private-reply flag.

`async def on_message(message)` Reacts only to messages wrapped in double backticks whose content is `restart`, `shutdown`, `get-env` or `show-env` (with an optional argument), and only from devs. Everyone else is ignored silently.

`async def _systemctl(action)` Runs `systemctl <action> SERVER_musicbots` without blocking. Change `SERVICE_NAME` if your unit has another name.

`async def _delayed_systemctl(action, delay=2.0)` Waits so the reply reaches Discord, then runs `_systemctl`.

`async def _dev_power(message, cmd, arg)` Body of `restart` / `shutdown`. `full` runs `systemctl`. No argument (or `all` / `0`) targets every worker; a number targets one. Restart waits up to 15 s per worker for its socket to answer; shutdown marks workers so the watcher doesn't revive them. The reply is edited with a per-worker result.

Slash command helpers

`async def _guard(interaction)` First step of the slash commands, before deferring so refusals are always private. Refuses commands outside a server, in a non-whitelisted server, or outside the control channel.

`def _text_channel_id(interaction)` Where the worker posts now-playing messages: the control channel when set, else the channel the command was used in.

`def _user_vc_id(interaction)` The voice channel the command author is in, or `None`.

`async def _fail(interaction, msg, embed=None)` Sends a private error follow-up.

`async def _pick_for_play(interaction, worker_arg)` Chooses `(worker, voice channel)` for `/play`, `/playlist` and `/autoplay`, or replies with an error and returns `None`. Runs under `_pick_lock` after `_resync_all`, and only considers workers that are in this server. - With the `worker` option: that worker. If it already has a session here, you must be allowed to control it; otherwise it joins its fixed channel or yours, unless another worker is already playing there. - When you are in a voice channel: the worker already there (if you are allowed to control it, otherwise the lock message), else a free worker fixed to your channel, else a free worker with no fixed channel, else "No bots available". - When you are in no voice channel: a free worker that has a fixed channel. - A newly chosen worker is reserved for you before the lock is released.

`async def _pick_for_control(interaction, worker_arg, for_stop=False)` Chooses `(worker, session)` for `/stop` and `/control` after a resync. With the `worker` option, that worker's session here. Otherwise the session in your voice channel, or the single session here that you can control. Refuses with the lock message when you are not allowed; with `for_stop`, Manage Server members are also allowed.

`async def _dispatch(interaction, cmd, worker, eph=False, timeout=30.0)` Sends a command to a worker and posts the result as a track/playlist embed, a short feedback embed or an error. Returns the worker's reply.

`async def _start_playback(interaction, op, eph_key, worker_arg, timeout=30.0, **extra)` Shared body of `/play`, `/playlist` and `/autoplay`: defers (private if the key says so), picks a worker, sends the operation with the user id and text channel, clears the reservation afterwards and stores the session snapshot from the reply.

Slash commands

`async def slash_play(interaction, query, source=None, worker=0)` `/play`. Uses `P_EPH`. Sends `search_and_play` with the chosen or server-default source.

`async def slash_playlist(interaction, query, source=None, worker=0)` `/playlist`. Uses `PL_EPH`. Sends `search_and_playlist`.

`async def slash_autoplay(interaction, worker=0)` /autoplay . Uses `AP_EPH` . Sends `autoplay_start` with a 60 s timeout (finding the first song can take a while).

`async def slash_stop(interaction, worker=0)` /stop . Uses `S_EPH` . Picks with `for_stop=True` , sends `stop` , and marks the session gone only when the stop succeeded.

`async def slash_control(interaction, worker=0)` /control . Uses `CC_EPH` . Picks the session, builds the panel and sends it.

`async def slash_hctest(interaction)` /hctest . Uses `HC_EPH` . Resyncs and pings every worker, then shows the controller status (latency, busy workers here, server count) and per worker: Discord and Lavalink state, whether it is busy here (with channel, controller and queue length), free or not in this server, how many servers it is active in, and its fixed channel.

`async def slash_settings(interaction)` /settings . Needs Manage Server or dev. Skips the control-channel check so admins can always fix it. Sends the private settings panel.

`async def slash_purge(interaction, limit=20)` /purge . Needs Manage Messages or dev. Deletes bot messages among the last `limit` messages and posts the count.

Control panel interface

`def _truncate(text, n=80)` Shortens text to n characters with an ellipsis.

`class QueueSelect(discord.ui.Select)` The queue dropdown, one page of up to 25 tracks. Choosing a track only selects it for Jump To / Remove. Options are keyed by the song's `qid` , so the selection stays on the same song even when the queue shifts; the number shown is its current position. - `__init__(self, view, queue, page, total_pages, current_title, selected_idx=None)` — Builds the options (🎵 for autoplay picks), an empty-queue placeholder and the placeholder text. - `callback(self, interaction)` — Stores the selection and refreshes the panel.

`class JumpToButton` — 🎵 Jump To, disabled until a track is selected. `callback` sends `jump_to` , reports the new track and refreshes from page 0.

`class RemoveButton` — 🗑 Remove, disabled until a track is selected. `callback` sends `remove_from_queue` and refreshes the same page.

`class ClearSelectionButton` — ✕ Clear. `callback` refreshes with no selection.

`class AutoplayButton` — 🎵 Autoplay / 🟢 Autoplay ON (green). `callback` sends `toggle_autoplay` .

`class ModeButton` — 🔒 Control: Me / 🔒 Control: All (green). `callback` sends `set_mode` with the other mode. Only the controller or a dev can press it (checked in `ControlView.interaction_check`).

`class PrevPageButton` , `class NextPageButton` — Move one queue page back/forward (disabled at the ends).

`class PageLabelButton` — Disabled "Page x / y" label.

`class ControlView(discord.ui.View)` The panel (15 minute timeout). Rows as in section 3. -

`__init__(self, worker, guild_id, queue, current_title, page=0, muted=False, loop=False, autoplay=False, mode="me", selected_idx=None)` — Stores the state and adds the dropdown and dynamic buttons. - `interaction_check(self, interaction)` — Runs before every button/dropdown. Refuses (privately) when the session has ended or the user isn't allowed (section 4). The mode button is controller/dev only; the Stop button is also allowed for Manage Server. -

`_dispatch(self, interaction, op, timeout=15.0, **extra)` — Shared handler: sends the operation, stores any session snapshot, shows feedback, and refreshes the panel after skip, restart, mute, loop, autoplay and mode changes. Returns the reply. - `rw10` , `rw5` , `ff5` , `ff10` — Seek -10/-5/+5/+10 s. -

`btn_loop` — `toggle_loop` . - `btn_backward` — backward (restart the track). - `btn_pause` — `pause_resume` . - `btn_stop` — `stop` ; only when it succeeds does it mark the session gone and disable the panel. - `btn_skip` — `skip` (45 s timeout, since it may need to fetch autoplay picks). - `btn_mute` — `toggle_mute` (hides now-playing messages; audio keeps playing).

`async def _build_control_view(worker, guild_id, page=0, selected_idx=None)` Fetches `get_queue`, stores the session snapshot, clamps the page, drops a selection that no longer exists, builds the view, colours the Mute/Loop buttons and builds the "Playback Controls — Worker N" embed (current track, flags, queue range, and who controls it).

`async def _refresh_control_panel(interaction, ctrl, page=0, selected_idx=None)` Rebuilds the panel and edits the message in place. Errors are only logged.

Settings panel interface

`def _settings_embed(guild, worker_idx, note=None)` The settings embed: optional result note, control channel, each worker's fixed channel (marks the one being edited; workers not in the server are noted), default source and private-reply list.

`def _channel_default(channel_id)` The pre-selected value for a channel picker, or nothing.

`class SettingsView(discord.ui.View)` The `/settings` panel (10 minute timeout). Every change saves immediately and redraws the panel. - `__init__(self, invoker_id, guild, worker_idx)` — Builds the pickers with the current values selected. - `interaction_check(self, interaction)` — Only the person who ran `/settings` may use it. - `_redraw(self, interaction, note=None, worker_idx=None)` — Replaces the panel with a fresh one showing `note`. - `_on_cc` — Sets or clears the control channel. - `_on_worker` — Switches which worker's fixed channel is being edited. - `_on_vc` — Sets or clears that worker's fixed channel; refuses a channel already fixed to another worker; notes when the worker is playing elsewhere (applies from its next play). - `_on_eph` — Saves the private-reply flags. - `btn_source` — Cycles the default source Spotify → YouTube Music → SoundCloud. - `btn_clear_cc`, `btn_clear_vc` — Clear the control channel / the selected worker's fixed channel. - `btn_done` — Shows "Settings saved" and removes the controls.

Supervision and start-up

`async def _watch_worker(w)` Per-worker loop every 5 s: a dead worker (or one that doesn't answer a ping when the controller has no process handle) has its state cleared and is restarted after 3 s, unless it was shut down on purpose. Every 30 s it also resyncs the worker, as a backstop in case a state event was lost.

`async def _run(stop_event=None)` Main coroutine. Migrates the old settings, starts the event server and the main bot, waits 5 s, creates one `WorkerProcess` per worker token, and for each either adopts an already-running worker (if its socket answers `sync`) or starts it, plus its watcher. Then waits for the stop signal and shuts everything down.

`def _sig()` SIGINT/SIGTERM handler in `--server` mode; sets the stop event.

7. worker.py reference (worker)

Module level: the worker index comes from `--index N` (falling back to a number at the end of the file name), its token is `tokens[N]` from `server_settings.json`, and the Lavalink address and password come from the same file. `GROQ_API_KEY` is read from `.env`. All playback state lives in `Session` objects in `sessions` (server id → `Session`).

Session

class Session One server's playback on this worker.

Field	Meaning
<code>guild_id</code> , <code>channel_id</code>	Server and voice channel.
<code>controller_id</code> , <code>mode</code>	Who started playback; "me" or "all".
<code>text_channel</code> , <code>announce</code>	Where now-playing messages go; False when playback was started privately (no channel posts).
<code>queue</code>	List of <code>QueueItem</code> s: (track, source label, found-via, requester id) plus a stable qid.
<code>fallbacks</code> , <code>origin</code>	Alternate candidates per track; resolved track → original track.
<code>skipping</code> , <code>muted</code> , <code>loop</code>	Skip guard, hidden now-playing messages, repeat current track.
<code>autoplay</code> , <code>autoplay_source</code> , <code>autoplay_user</code> , <code>autoplay_task</code>	Autoplay on/off, "session" or "algo", whose saved songs, background refill task.
<code>fail_streak</code>	Failed tracks in a row.
<code>current_info</code> , <code>current_meta</code> , <code>current_requester</code>	Real title/artist of the playing track, its algo record, who requested it.
<code>idle_task</code> , <code>auto_paused</code>	Idle countdown; paused because the channel emptied.
<code>connecting</code> , <code>closing</code>	Guards so our own connect/disconnect isn't mistaken for being kicked.
<code>history</code>	The last 25 played tracks (autoplay seeds and duplicate filter).

- **snapshot(self)** — The small dict sent to the controller: `channel_id`, `controller_id`, `mode`, `autoplay`, `text_channel_id`.

class QueueItem(tuple) A queued song: (track, source label, found-via, requester id) with a unique, never-reused qid. The control panel refers to songs by qid, not by position, because positions shift whenever a song ends or autoplay adds picks (which used to make Jump To / Remove hit the song *after* the one chosen).

def _queue_pos(sess, qid) Current position of the song with that qid, or None when it already played or was removed.

Groq filters

async def _is_altered(title) Asks the Groq model (openai/gpt-oss-20b) whether a title is an altered version (sped up, slowed, remix, cover, live, instrumental, ...). 4 s limit; errors count as clean.

async def _user_wants_altered(query) Asks whether the search text explicitly asks for an altered version, so a search for a remix isn't filtered. Errors count as "no". Queries containing an obvious word (sped up, slowed, reverb, nightcore, 8d, bass boost, lofi, remix, cover, karaoke, instrumental, mashup, tiktok — `_ALTERED_ASK`) count as asking without calling the AI, so a Groq timeout can't filter out a version you named.

Altered versions are only played when you ask for them: the main search checks every candidate with `_is_altered`; matching a Spotify/Apple Music song to YouTube skips uploads whose title looks altered (unless the original's does); autoplay skips them by title. Only if *every* candidate is altered does the search fall back to the best one.

`async def _ai_pick(query, cands)` One AI call per search. It gets the search text and the top `AI_PICK_CANDIDATES` (8) results (title, channel, length, views) and returns JSON: `wants_altered` (did the search explicitly ask for a sped up / slowed / remix / cover ... version; genre, artist, language or mood words don't count), `altered` (which results are non-original versions, including mashups, montagens and medleys) and `pick` (the result that is the song the user means). Returns None when Groq fails or times out (8 s); the search then uses the title rules.

The model is a reasoning model: its thinking counts toward `max_tokens`. All Groq calls use `_GROQ_ARGS` (400 tokens, low reasoning effort). Earlier versions allowed 3 tokens, so every answer came back empty and was read as "not altered", which is why altered versions used to slip through.

`async def _groq_similar(seeds, count=5)` Autoplay fallback: asks the model for `count` real songs similar to the seeds, one "Title - Artist" per line, and returns them as search queries. Errors return an empty list.

Source helpers and scoring

`def _source_label(hint, query)` Display name of the source (Spotify, SoundCloud, YouTube, Apple Music) from the URL, else from the `sp / sc / yt` hint.

`def _is_local(track)` True for Spotify local files and other non-streamable URIs; they are skipped in playlists.

`def _fold(text)`, `def _words(text)` Lower-case and strip accents ("Tântrico" → "tantrico"); split into words of 2+ characters. All matching is accent-insensitive.

`def _track_score(track, query="", ref=None)` Relevance of a search candidate. Each query word in the title +40, in the author +20. With a reference track (the Spotify match) each of its words +15 in the title or +5 in the author, and the length difference +60 (≤ 3 s), +25 (≤ 8 s) or -40 (> 20 s). "Official" +10; altered-version words -20 (`_ALTERED_QUICK`). Medleys (titles with two or more "/") -60; tracks over 10 minutes -30 unless the query asks for a mix/full/album/hour/live; mashups ("Song A x Song B") -50 and snippets ("best part") -30 unless the query has them.

`async def _taste(guild_id, user_id)` The requester's saved songs summarised for searching: plays per YouTube id and per artist (cached 60 s).

`def _taste_bonus(track, taste)`, `def _taste_note(track, taste)` The personal score boost for a result, and the note shown to the AI (section 5).

`def _popularity(meta)` Popularity points from YouTube stats: $8 \times \log_{10}(\text{views} + 1) + 4 \times \log_{10}(\text{likes} + 1)$. About 99 for a billion-view hit, 48 for 50k views, 0 for an unwatched upload. Relevance still dominates: a song matching one fewer query word (-40) needs roughly 100× more views to win.

Embeds and messages

`def _track_embed(track, action, source_label, queue_pos=0, search_path="")` Now Playing / Added to Queue embed.

`def _now_playing_embed(track, source_label, search_path="")` Shortcut for the Now Playing embed sent on track changes.

`async def _send_status(channel, embed)` Sends an embed and deletes it after 10 s (`_STATUS_DELETE_DELAY`).

`async def _delete_after(msg, delay)` Deletes a message after a delay, ignoring errors.

`async def _set_vc_status(guild_id, channel_id, title)` Sets the voice channel status to  Playing - <title> (max 500 characters), or clears it with None. Needs the Set Voice Channel Status permission; failures are only logged.

Track bookkeeping

def _track_key(track) Stable key: identifier, else URI, else "title:author".

def _remember_alternates(sess, track, candidates, source_label, search_path) Stores the other search candidates as fallbacks for the chosen track in that session (no-op without a session, e.g. for autoplay AI searches).

def _track_meta(origin, yt_src) The algo record for a song: key (YouTube id when known, else URL), title, author, uri, yt_id.

Events to the controller

def _notify(guild_id) Queues a state event with the session's snapshot (or None when it ended).

async def _event_sender() Sends queued events to `.main.sock` one at a time, so they arrive in order. A lost event is harmless because the controller resyncs.

def _bg(fn, *args) Runs a blocking DB function in a thread in the background and logs errors.

yt-dlp resolver and playback

async def _ytdlp_run(video_url) Runs `python -m yt_dlp -g -f bestaudio/best ...` and returns the direct stream URL (nothing is downloaded). 20 s limit.

async def _ytdlp_url(video_url) Cached (1 h) and de-duplicated wrapper around `_ytdlp_run`.

async def _yt_meta(video_url) One `yt-dlp -j` call that returns a video's views, likes and direct stream URL. Cached (1 h) and de-duplicated; the stream URL is also put in the resolver cache, so the chosen track starts without a second yt-dlp run.

async def _rank_by_popularity(scored) Looks up `_yt_meta` for the `P0P_CHECK` (4) most relevant candidates at once (skipping obviously altered titles unless you asked for one) (about 2 s on the Pi), adds `_popularity` to their relevance, logs each line (relevance + popularity = total, views, likes) and returns all candidates re-sorted.

async def _resolve(track) Turns a track into something Lavalink can play. A YouTube video is tried three ways before giving up on it: the cached yt-dlp stream URL, a freshly fetched one (YouTube sometimes refuses a cached URL), then Lavalink's own YouTube plugin. Spotify/Apple Music tracks are matched to the best YouTube Music result first (never to an altered upload unless the original is one); non-YouTube sources play as they are; YouTube tracks are resolved with yt-dlp. Returns (playable, original track, YouTube track or None) or None.

def _prefetch_next(sess) Resolves the next queued YouTube track in the background.

def _set_current(sess, origin, yt_src, requester) Records the playing track's real title, its algo record and requester, and adds it to the history.

def _after_start(sess) After a track starts: sets the voice channel status, cancels the idle timer, prefetches the next track, and starts an autoplay refill when autoplay is on and the queue is empty.

async def _play(sess, player, track, requester) Plays a track, trying it and then its alternates (at most `MAX_RESOLVE_ATTEMPTS`, 2). Alternates are only other uploads of the same song (`_same_song`). When nothing resolves it hands the original to Lavalink anyway. Returns the track actually playing, which is what the reply, the Now Playing message and the channel status show.

def _account(sess, played_ms=None, finished=False, skipped=False) Feeds the outgoing track into its requester's saved songs: a listen when finished or played ≥ 30 s, a skip when skipped earlier, nothing otherwise. Writes happen in the background.

def _enqueue(sess, item) Adds a user's track ahead of any autoplay picks and returns its 1-based position.

`async def _advance(sess, player, announce=True)` Plays the next queued track, refilling from autoplay first when the queue is empty and autoplay is on. With nothing left it clears the voice channel status and starts the idle timer.

Autoplay

`def _same_song(a, b)` True when two results are uploads of the same song: same normalised title and either the same artist or a length within 8 s. Used to limit fallbacks, so a same-named song by someone else is never played in place of the one picked.

`def _norm_title(title)` Lower-cases a title and strips brackets and words like "official", "lyrics", "video", so different uploads of the same song compare equal.

`def _excluded_sets(sess)` Video ids and normalised titles already played in the session or queued.

`def _usable_rec(t, ids, titles)` True for a pick that isn't excluded, isn't an altered version and is 1–10 minutes long.

`async def _yt_id_for(seed)` The seed's YouTube id, looking it up on YouTube Music when unknown.

`async def _mix_for(seed)` Loads YouTube's radio mix for the seed and returns its tracks without the seed itself.

`def _weighted_pick(rows, k)` Picks up to k distinct saved songs at random, weighted by score.

`async def _autoplay_seeds(sess)` Seeds for the next picks: weighted saved songs of `autoplay_user` in `algo` mode, else the most recent songs of the session (`autoplay_seed_count`).

`async def _recommend(sess, count)` Finds up to `count` picks: in `algo` mode sometimes a saved song directly, then one Mix pick per seed, then the Groq fallback if nothing was found.

`async def _autoplay_refill(sess)` Queues the picks (credited to the autoplay user or controller) if the queue is still empty and the session still exists.

`def _kick_autoplay(sess)` Starts a background refill unless one is running.

`async def _autoplay_fill(sess)` Starts (or joins) a refill and waits for it.

Idle and session lifecycle

`def _start_idle_timer(sess)`, **`def _cancel_idle_timer(sess)` Start (if not running) or cancel the session's idle countdown.**

`def _humans_in_vc(sess)` True when someone who isn't a bot is in the session's voice channel.

`async def _idle_countdown(sess)` After `IDLE_TIMEOUT` (180 s) ends the session, unless music is actively playing (not auto-paused) with people in the channel.

`async def _end_session(guild_id, disconnect=True)` Removes the session, cancels its timers and autoplay, records the current track's listen, clears the voice channel status and leaves the channel (when `disconnect`), and notifies the controller.

`def _claim(guild_id, channel_id, user_id)` Returns the server's session (creating it with `user_id` as controller when there is none). Raises when the worker already plays in a different channel of that server.

`async def _do_shutdown()` After the `shutdown_graceful` reply: runs `_shutdown` and stops the event loop.

`def _get_player(guild_id)` The wavelink player (voice client) in a server, or None.

`def _session_for_player(player)` The session belonging to a player's server, or None.

Discord and Lavalink events

`async def _enforce_whitelist(guild)` Leaves a non-whitelisted server silently (the main bot posts the notice).

async def on_ready() Enforces the whitelist; on the **first** ready only, starts the event sender, the Lavalink connection and the IPC server (Discord reconnects fire **on_ready** again, and these must not start twice).

async def on_guild_join(guild) Enforces the whitelist.

async def on_wavelink_node_ready(payload) , **async def on_wavelink_node_disconnected(payload)** Track and log the Lavalink connection (wavelink reconnects by itself).

async def on_wavelink_track_end(payload) For the session of that server: a **finished** track resets the fail streak and counts as a listen. **loadFailed** , **replaced** , **stopped** and **cleanup** are ignored (already handled elsewhere), as is anything during a skip. Otherwise it re-queues the track when loop is on and advances.

async def on_wavelink_track_exception(payload) Logs the error and counts the failure. After **MAX_FAIL_STREAK** (3) in a row it posts "Playback failed", clears the queue, turns autoplay off, stops and starts the idle timer. Otherwise it tries the failed track's next alternate, or advances.

async def on_voice_state_update(member, before, after) - The worker itself left voice (kicked or disconnected by someone) → the session ends, so the worker is free again. - The worker was dragged to another channel → the session follows it. - The last person left the session's channel → playback pauses and the idle timer starts. - Someone came back → playback resumes and the idle timer stops.

Connections

async def _connect_lavalink() Connects to Lavalink, retrying forever (5 s more per attempt, at most 30 s).

async def _connect_vc(sess) Returns a player connected to the session's channel: reuses, moves or connects with **VC_RETRIES** attempts of **VC_TIMEOUT** each. Raises a readable error when the worker isn't in the server or the channel is invalid.

Search

async def _sp_lookup(query) Spotify search (**spsearch:**) for clean title/artist/length. Of the top 5 results it takes the one containing the most of the query's words, and only if it contains at least half of them (**SP_MIN_COVERAGE**); otherwise None. Spotify's first hit can be unrelated, and Spotify errors also give None.

async def _am_load(url) Loads an Apple Music track, album or playlist through LavaSrc.

async def _yt(query, count=8) , **async def _sc(query, count=5)** YouTube Music / SoundCloud search through Lavalink.

async def _first_clean(candidates) First candidate the Groq filter doesn't consider altered.

async def _search(query, source, sess) Finds the track for a query and returns (**track** , **source label** , **found-via**) . Apple Music and other URLs load directly. **sc** searches SoundCloud and prefers a non-altered result. **sp** (default) runs the Spotify lookup, the altered-version check and a YouTube Music search for the exact query at the same time; if Spotify gave a matching song, YouTube Music is also searched with its clean title/artist and both result lists are merged. **yt** skips Spotify. Candidates are ranked by relevance plus popularity (**_rank_by_popularity**), then **_ai_pick** decides. Its answer is checked: unless an altered version was asked for, results the AI or obvious title words mark as altered are never picked; and the pick must match the search about as well as the best result (relevance within 40), so a slowed version of a *different* song can't win. If the AI fails, obvious title words decide instead. Remaining allowed results become the fallbacks. Only if every result is altered does it try preferred lyric channels, then the best-ranked result. An explicitly requested altered version skips the filter. Alternates are stored on the session.

async def _search_playlist(query, source) Loads a playlist (Apple Music, URL, SoundCloud search or Spotify search) and returns (**playable tracks** , **name** , **source label** , **skipped local files**).

IPC server and shutdown

def _track_reply(track, embed_type, lbl, path, **extra) The reply fields the controller turns into a track embed.

`async def _handle_connection(reader, writer)` Handles one IPC connection: reads one JSON line, runs the operation (section 9) and writes one JSON reply. Operations that can start playback go through `_claim` and `_start_op`; if a new session ends up playing nothing (search failed, connect failed), it is ended again so the worker is free. Any exception becomes an error reply.

`async def _start_op(sess, op, cmd, uid, reply)` The `search_and_play / queue_track`, `search_and_playlist` and `autoplay_start` operations.

`async def _ipc_server()` Starts the Unix socket server on `.worker<N>.sock`.

`async def _shutdown()` Closes the Discord client and deletes the socket file.

8. db.py and appsettings.py reference

db.py

All functions are blocking SQLite calls (WAL mode, 10 s busy timeout), safe to use from the controller and all workers at once. The worker calls them in a background thread.

def _connect(path) Opens a SQLite file (creating its folder) with row access by name and WAL mode.

def _server_con() Opens `database/server/servers.db` and creates or upgrades the `guild_settings` table (`guild_id`, `cc_id`, `vcw` JSON, `eph` JSON, `src`, `updated_at`).

def get_guild(guild_id) A server's settings as `{cc_id, vcw, eph, src}`; defaults when the server has none.

def save_guild(guild_id, cfg) Inserts or updates a server's settings.

def guild_exists(guild_id) True when the server has stored settings.

def _algo_con(guild_id) Opens `database/algo/<guild_id>.db` and creates the `user_tracks` table (`user_id`, `track_key`, `title`, `author`, `uri`, `yt_id`, `plays`, `skips`, `last_played`, `added_at`; one row per user and song).

def record_play(guild_id, user_id, meta, max_kb) Adds a listen (inserting the song with 1 play, or +1 play and refreshing its details), then evicts.

def _evict(con, user_id, keep_key, max_kb) Removes the user's lowest-score songs (oldest play breaks ties) until they are within the storage cap, never removing `keep_key`. There is no song-count limit.

def record_skip(guild_id, user_id, key) +1 skip on a saved song; does nothing for unsaved songs.

def user_tracks(guild_id, user_id, limit=100000) The user's saved songs, best score first, with a `score` field.

def user_stats(guild_id, user_id) (song count, approximate bytes) for a user.

appsettings.py

def get() Returns `server_settings.json` merged over the defaults, re-reading the file only when it changed. A broken file keeps the previous values and logs the error.

def require_startup() Used at start-up by the controller and workers. Exits with a clear message when the file is missing, has fewer than 2 tokens or no Lavalink URI.

def is_dev(user_id) True when the user id is in `dev_ids`.

def is_whitelisted(guild_id) True when the whitelist is off or the server is listed.

9. IPC operations and state events

Controller → worker

The controller sends one JSON object per connection to `.worker<N>.sock`, with `op` plus `guild_id`, `channel_id`, `text_channel_id` and `user_id` where relevant. The worker answers with one JSON line: `status` (`ok` or `error`), `message`, and extra fields. Replies that change a session include `session` (its snapshot).

op	What the worker does	Reply
ping	Reports its health (and its session in <code>guild_id</code> , if given).	pong with <code>discord_ok</code> , <code>discord_ms</code> , <code>lavalink_ok</code> , <code>lavalink_uri</code> , <code>sessions</code> (count), <code>session</code> , <code>queue_len</code>
sync	Reports its full state.	synced with <code>user_id</code> , <code>guilds</code> , <code>lavalink_ok</code> , <code>sessions</code> (server id → snapshot)
search_and_play, queue_track	Claims the session, searches, connects, then plays or queues (ahead of autoplay picks).	playing or queued with <code>title</code> , <code>author</code> , <code>uri</code> , <code>artwork</code> , <code>duration</code> , <code>source_label</code> , <code>search_path</code> , <code>queue_pos</code> , <code>session</code>
search_and_playlist	Loads the playlist; plays the first and queues the rest, or queues all.	playing_playlist or queued_playlist with <code>pl_name</code> , <code>count</code> , <code>title</code> , <code>artwork</code> , <code>source_label</code> , <code>skipped</code> , <code>session</code>
autoplay_start	Checks the user has saved songs, turns on autoplay from them; starts playing, or switches an already-playing session.	playing (as above) or <code>autoplay_on</code>
stop	Ends the session (records the listen, clears the status, leaves).	stopped
pause_resume	Pauses or resumes.	paused or resumed
skip	Records a listen (≥ 30 s played) or a quick skip, stops the track and advances (refilling autoplay if needed).	skipped
backward	Seeks to the start.	restarted
seek	Moves by <code>delta_ms</code> , clamped to the track.	seeked with <code>delta_ms</code>
remove_from_queue	Deletes the song with <code>qid</code> . Error if it already played or was removed.	removed with <code>title</code> , <code>queue_remaining</code>
get_queue	Returns the current track and the queue: each entry has <code>title</code> , <code>author</code> , <code>index</code> (position), <code>qid</code> and an <code>autoplay</code> flag.	queue with <code>current</code> , <code>queue</code> , <code>muted</code> , <code>loop</code> , <code>autoplay</code> , <code>session</code>
toggle_mute	Hides or shows now-playing messages.	muted or unmuted
toggle_loop	Repeats the current track or not.	loop_on or loop_off
toggle_autoplay	Turns autoplay on (needs played history unless in algo mode; starts playing if idle) or off (drops queued picks).	autoplay_on or autoplay_off, <code>session</code>
set_mode	Sets mode to <code>me</code> or <code>all</code> .	mode_me or mode_all, <code>session</code>

op	What the worker does	Reply
jump_to	Records a listen or quick skip for the current track, drops the queue up to the song with <code>qid</code> and plays it. Error if it already played or was removed.	jumped with <code>title</code> , <code>author</code> , <code>queue_remaining</code>
shutdown_graceful	Ends every session, replies, then exits.	shutdown_graceful
drain	Graceful restart: blocks new playback; each session finishes its song, plays the message at <code>message_url</code> and leaves.	draining with <code>remaining</code>
drain_force	Restart timeout: cuts songs still playing and plays the message now.	forced with <code>remaining</code>

Session operations on a server where the worker has no session reply "Nothing playing.". Any other op returns "Unknown op".

Worker → controller

Sent to `.main.sock`, one JSON line per connection, in order:

```
{"op": "state", "index": 2, "guild_id": 123, "session": {"channel_id": 456, "controller_id": 789, "mode": "me", "autoplay": false, "text_channel_id": 111}}
```

`session` is `null` when the session ended. Sent when a session is created or ends, and on autoplay, mode and channel changes.

10. Tunable constants and files

Name	File	Value	Meaning
<code>SERVICE_NAME</code>	main.py	<code>SERVER_musicbots</code>	systemd unit used by <code>restart full / shutdown full</code> .
<code>DRAIN_TIMEOUT</code>	main.py	300 s	Graceful restart: longest wait for current songs before they are cut.
<code>DRAIN_MESSAGE_MAX</code>	main.py	130 s	Graceful restart: extra wait for the shutdown messages after the timeout.
<code>SHUTDOWN_MESSAGE_MAX</code>	worker.py	120 s	A worker leaves this long after starting the message, even if it never reports finishing.
<code>PENDING_TTL</code>	main.py	90 s	How long a <code>/play</code> reservation holds a worker without confirmation.
<code>PAGE_SIZE</code>	main.py	25	Queue entries per dropdown page (Discord limit).
ControlView timeout	main.py	900 s	Control panel buttons stop working after 15 minutes.
SettingsView timeout	main.py	600 s	Settings panel stops working after 10 minutes.
<code>VC_TIMEOUT</code> , <code>VC_RETRIES</code>	worker.py	60 s, 3	Time and attempts for joining or moving to a voice channel.
<code>IDLE_TIMEOUT</code>	worker.py	180 s	Idle time before the worker leaves.
<code>YTDLP_TIMEOUT</code>	worker.py	20 s	Maximum time for one yt-dlp lookup.
<code>URL_CACHE_TTL</code>	worker.py	3600 s	How long a resolved stream URL is reused.
<code>MAX_RESOLVE_ATTEMPTS</code>	worker.py	2	Candidates tried per play before handing the original to Lavalink.
<code>MAX_FAIL_STREAK</code>	worker.py	3	Failed tracks in a row before the worker stops and reports.
<code>POP_CHECK</code>	worker.py	4	Candidates whose YouTube views/likes are looked up per search.
<code>POP_VIEW_W</code> , <code>POP_LIKE_W</code>	worker.py	8, 4	Popularity points per 10× views / likes.
<code>SP_MIN_COVERAGE</code>	worker.py	0.5	Share of the query's words a Spotify match must contain to be used.
<code>PLAY_COUNT_MS</code>	worker.py	30 s	Listening time for a song to count as a listen.
<code>HISTORY_LEN</code>	worker.py	25	Played tracks remembered per session.
<code>AUTOPLAY_BATCH</code>	worker.py	2	Autoplay picks queued per refill.
<code>ALGO_DIRECT_CHANCE</code>	worker.py	0.3	<code>/autoplay</code> : chance of replaying a saved song instead of a related one.
<code>_STATUS_DELETE_DELAY</code>	worker.py	10 s	Lifetime of the now-playing messages.
<code>AI_PICK_CANDIDATES</code>	worker.py	8	Search results shown to the AI per search.
<code>_GROQ_ARGS</code>	worker.py	400 tokens, low reasoning	Settings for every Groq call (the AI pick allows 1200 tokens).

File	Created by	Purpose
server_settings.json	you	Tokens, Lavalink, devs, whitelist, algo and autoplay limits. Secret, git-ignored.
server_settings.json.example	repo	Template for the above.
.env	you	GR0Q_API_KEY . Secret, git-ignored.
database/server/servers.db	controller	Per-server settings from /settings .
database/algo/<server id>.db	workers	Every user's saved songs for that server.
.worker<N>.sock	worker	Socket the controller uses to talk to worker N.
.main.sock	controller	Socket the workers send state events to.
shutdown.mp3	you	Message played before a restart (optional).
settings.json.migrated	controller	The old single-server settings after their one-time import.

To reset a server's settings, delete its row from `servers.db` (or the whole file to reset every server). To forget everyone's saved songs on a server, delete `database/algo/<server id>.db` .

11. Graceful restart

`systemctl restart SERVER_musicbots` (and `restart full`) or `systemctl stop (shutdown full)` no longer cut the music:

1. systemd sends the stop signal to the **main bot only** (`KillMode=mixed`).
2. The main bot blocks new commands ("restarting — try again in a few minutes") and sends `drain` to every worker. A worker that is draining refuses new playback ("hard block"), even after its own song finished.
3. Each worker, per server: - a song is playing → it finishes **that song**; the rest of the queue/playlist and autoplay are dropped; - paused or between songs → the message plays right away; - nobody in the voice channel → it leaves without a message.
4. After the song, the worker plays `shutdown.mp3` (next to `main.py`) and leaves when it ends (at most 2 minutes). Without the file, it just leaves.
5. The main bot waits until **every** worker is done, or `DRAIN_TIMEOUT` (5 minutes). At the timeout, songs still playing are cut and the message plays immediately (`drain_force`); then it waits for those messages.
6. Everything exits and systemd starts it again (restart) or leaves it stopped (stop).

A second stop signal (e.g. running `systemctl kill SERVER_musicbots` while it waits) skips the wait and goes straight to step 5. If nothing is playing, it restarts immediately.

The message is served to Lavalink over a private `http://127.0.0.1:<random port>/shutdown.mp3` by the main bot, so Lavalink must run on the same machine (its `http` source is enabled; its `local` file source is not needed).

Required service settings (in `/etc/systemd/system/SERVER_musicbots.service`):

```
KillMode=mixed
TimeoutStopSec=480
```

`TimeoutStopSec` must be longer than the 5-minute wait plus the message, otherwise systemd kills everything before it's done.

12. Deployment and service

The service file is `/etc/systemd/system/SERVER_musicbots.service`. It runs `venv/bin/python main.py --server` from `/home/jared/servers/musicbot_updated`, after `SERVER_lavalink`. A backup of the version that pointed at the old folder is at `~/servers/SERVER_musicbots.service.bak`.

Fresh install:

```
python3 -m venv venv
```

```
venv/bin/pip install -r requirements.txt
```

Then copy `server_settings.json.example` to `server_settings.json`, fill it in, create `.env` with `GROQ_API_KEY=...`, and start the service:

```
sudo systemctl restart SERVER_musicbots
```

Follow the logs (controller and every worker, prefixed `[Worker N]`):

```
journalctl -u SERVER_musicbots -f
```

Notes:

- Don't run a second copy (for testing) while the service is running: they share the same bot tokens.
- Slash commands are registered globally; new or changed commands can take a minute to appear (Ctrl+R in Discord shows them immediately).
- Spotify's Web API requires the owner of the Spotify developer app (configured in Lavalink's `application.yml`) to have an active Premium subscription. Without it, Spotify searches and `open.spotify.com` links fail; searches then fall back to YouTube Music only.
- The Message Content Intent is a privileged intent; beyond 100 servers Discord requires the bot to be verified for it.